Лабораторная работа №2

Бессонов Андрей Максимович

Содержание

1 Цель работы 5

2 Задание 6

3 Теоретическое введение 7

4 Выполнение лабораторной работы 9

5 Выводы 17

Список литературы 18

Список иллюстраций

4.1 Рис.1 10
4.2 Рис.2 11
4.3 Рис.3 12
4.4 Рис.4 13
4.5 Рис.5 14
4.6 Рис.6 14
4.7 Рис.7 15
4.8 Рис.8 16

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

2 Задание

Сделайте отчёт по предыдущей лабораторной работе в формате Markdown. В качестве отчёта просьба предоставить отчёты в 3 форматах: pdf, docx и md (в архиве, поскольку он должен содержать скриншоты, Makefile и т.д.)

3 Теоретическое введение

3.0.1 Системы контроля версий. Общие понятия

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких че-

ловек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом.

Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.

В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным.

Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

3.0.2 Примеры использования git

Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды git с различными опциями. Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым копированием или архивацией.

4 Выполнение лабораторной работы

В ходе работы мы выполнили все поставленные задачи:

4.0.1 Базовая конфигурация Git

Мы настроили глобальную конфигурацию Git: - Указали имя пользователя и email - Настроили корректное отображение UTF-8 символов - Задали имя начальной ветки `master` - Настроили параметры окончаний строк для кросс-платформенной работы

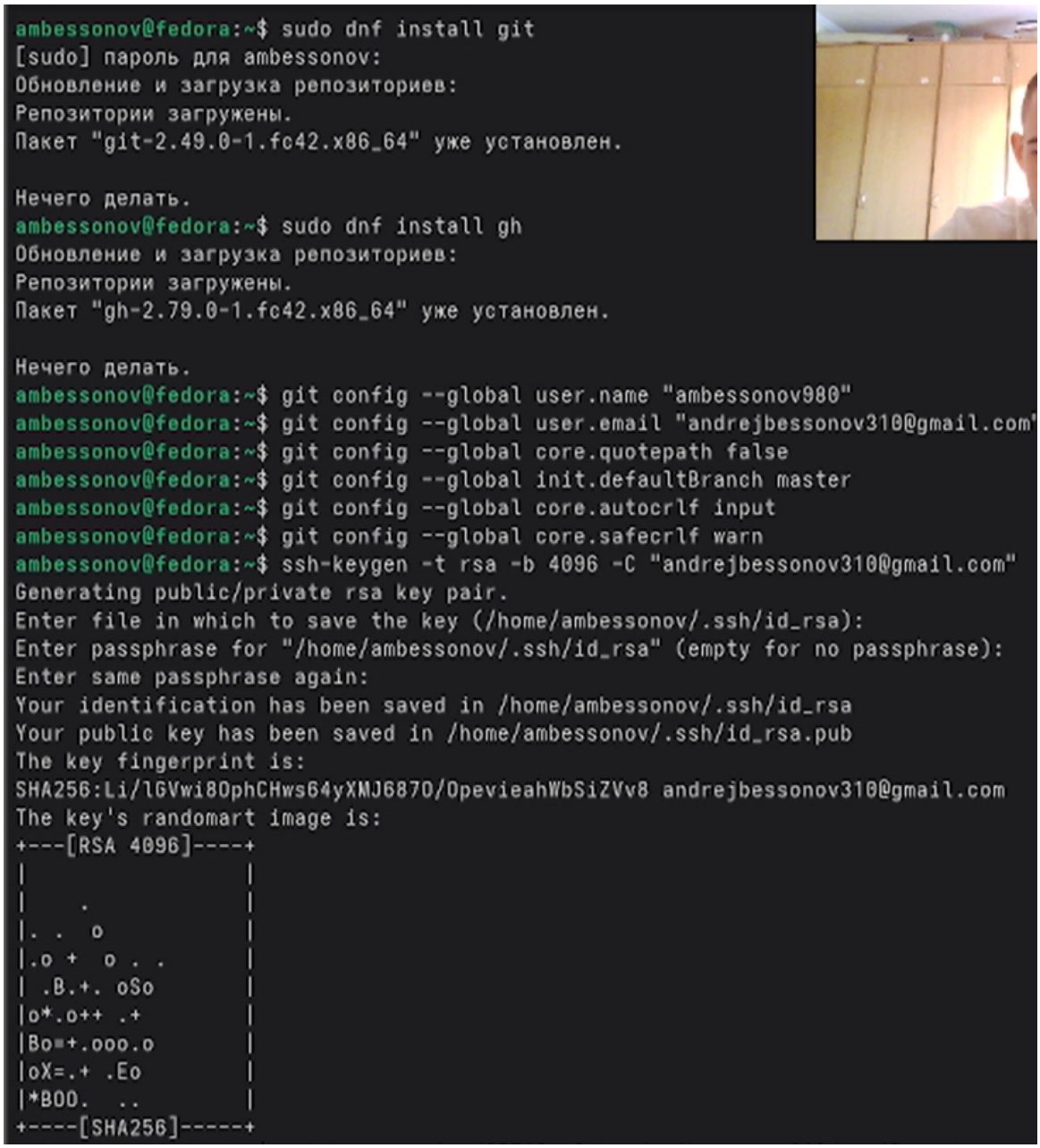

Рисунок 4.1: Рис.1

4.0.2 Создание ключей SSH

Мы сгенерировали ключи для безопасного подключения к GitHub: - Создали RSA ключ (4096 бит) - Создали Ed25519 ключ (современный алгоритм)

```
ambessonov@fedora:~$ ssh-keygen -t ed25519 -C "andrejbessonov310@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ambessonov/.ssh/id_ed25519):
Enter passphrase for "/home/ambessonov/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ambessonov/.ssh/id_ed25519
Your public key has been saved in /home/ambessonov/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:ik//PnLQUN5THR6CvcM9T4y3Aq+i4DD02BTyIorOUQ0 andrejbessonov310@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|            o. +.|
|           .. .+ o|
|   E .   o ...o+ |
|    = . . ..o+.o+|
| . + +  So  o..o+|
|o + *. .. .  o ..|
|o. +.+o  .  . .  |
|o . +o....o.     |
| o   ...o=+.     |
+----[SHA256]-----+
ambessonov@fedora:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.7; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
```

Рисунок 4.2: Рис.2

4.0.3 Создание PGP ключа

Мы создали ключ для подписи коммитов: - Сгенерировали GPG ключ с параметрами RSA and RSA (4096 бит) - Нашли отпечаток ключа - Экспортировали публичный ключ для добавления на GitHub

```
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: ambessonov980
Адрес электронной почты: andrejbessonov310@gmail.com
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "ambessonov980 <andrejbessonov310@gmail.com>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? O
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтроп
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтроп
gpg: сертификат отзыва записан в '/home/ambessonov/.gnupg/openpgp-revocs.d
C81B2651359.rev'.
открытый и секретный ключи созданы и подписаны.

pub   rsa4096 2026-02-23 [SC]
      B5413DF0EC26894C1BCA1E49CB007C81B2651359
uid                      ambessonov980 <andrejbessonov310@gmail.com>
sub   rsa4096 2026-02-23 [E]
```

Рисунок 4.3: Рис.3

4.0.4 Настройка подписей коммитов

Мы настроили автоматическую подпись всех коммитов: - Указали Git какой ключ использовать для подписи - Включили автоматическую подпись коммитов - Настроили программу для GPG

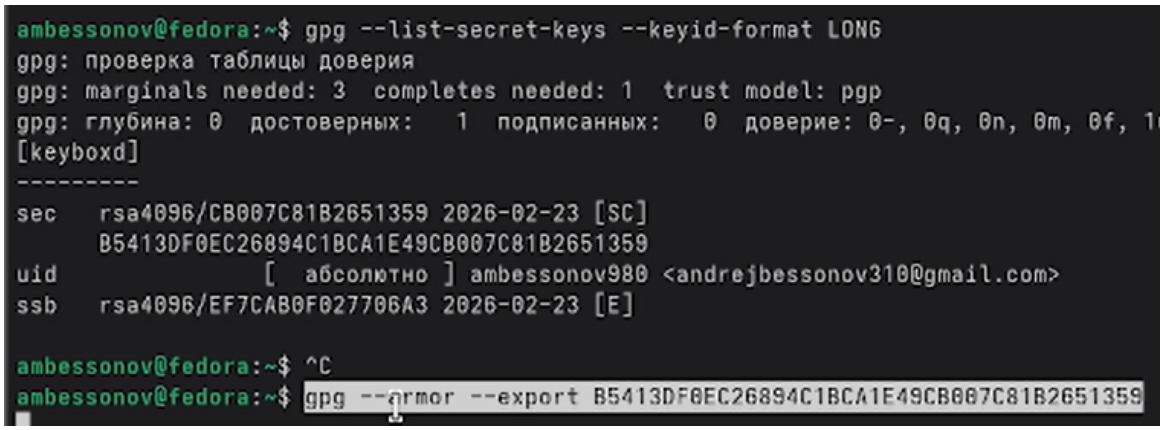

Рисунок 4.4: Рис.4

4.0.5 Регистрация на GitHub

Мы создали учетную запись на GitHub и заполнили основные данные

```
ambessonov@fedora:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIORKRa2TzAD1UaZ5LmZnzAPUXx6zmMHH82s3XW3fO6mz andrejbessonov310@gmail
.com
ambessonov@fedora:~$ ^C
ambessonov@fedora:~$ git config --global user.signingkey B5413DF0EC26894C1BCA1E49CB007C81B2651359
ambessonov@fedora:~$ ^[[200~git config --global commit.gpgsign true
bash: git: команда не найдена...
ambessonov@fedora:~$ ^C
^[[200~git config --global commit.gpgsign true
^[[201~ambessonov@fed
ambessonov@fedora:~$ git config --global commit.gpgsign true
ambessonov@fedora:~$ git config --global gpg.program $(which gpg2)
ambessonov@fedora:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/ambessonov/.ssh/id_ed25519.pub
? Title for your SSH key: GitHub CLI
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 8259-81B5
Press Enter to open https://github.com/login/device in your browser...

✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
✓ SSH key already existed on your GitHub account: /home/ambessonov/.ssh/id_ed25519.pub
✓ Logged in as ambessonov980
```

Рисунок 4.5: Рис.5

4.0.6 Создание локального каталога

Мы создали рабочее пространство для выполнения заданий: - Создали структуру каталогов `~/work/study/2022-2023/Операционные системы/os-intro` - Склонировали репозиторий на основе шаблона - Выполнили первичную настройку каталога курса

```
ambessonov@fedora:~$ mkdir -p ~/work/study/2026-2027/"Операционные системы"
ambessonov@fedora:~$ cd ~/work/study/2026-2027/"Операционные системы"
ambessonov@fedora:~/work/study/2026-2027/Операционные системы$ gh repo create study_2022-2023_os-intro -
-template=yamadharma/course-directory-student-template --public
✓ Created repository ambessonov980/study_2022-2023_os-intro on github.com
  https://github.com/ambessonov980/study_2022-2023_os-intro
ambessonov@fedora:~/work/study/2026-2027/Операционные системы$ git clone --recursive git@github.com:ambe
ssonov980/study_2022-2023_os-intro.git os-intro
Клонирование в «os-intro»...
The authenticity of host 'github.com (140.82.121.4)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Рисунок 4.6: Рис.6

4.0.7 Добавление ключей на GitHub

Мы привязали созданные ключи к нашей учетной записи: - Добавили SSH-ключ в настройки аккаунта - Добавили PGP-ключ в настройки аккаунта

```
ambessonov@fedora:~/work/study/2026-2027/Операционные системы$ cd os-intro
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ rm package.json
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ echo os-intro > COURSE
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ make
Usage:
  make <target>

Targets:
  list                            List of courses
  prepare                         Generate directories structure
  submodule                       Update submules

ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git add .
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git status
Текущая ветка: master
Эта ветка соответствует «origin/master».

Изменения, которые будут включены в коммит:
  (используйте «git restore --staged <файл>...», чтобы убрать из индекса)
        изменено:      COURSE
        удалено:       package.json

ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git commit -am 'feat(main): make
 course structure'
[master dd7631c] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 16 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 956 байтов | 478.00 КиБ/с, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:ambessonov980/study_2022-2023_os-intro.git
   9c960be..dd7631c  master -> master
ambessonov@fedora:~/work/study/2026-2027/Операционные системы/os-intro$
```

Рисунок 4.7: Рис.7

4.0.8 Первый подписанный коммит

Мы выполнили первый коммит с автоматической подписью: - Создали необходимые файлы и каталоги - Добавили файлы в отслеживание - Создали подписанный коммит - Отправили изменения на GitHub

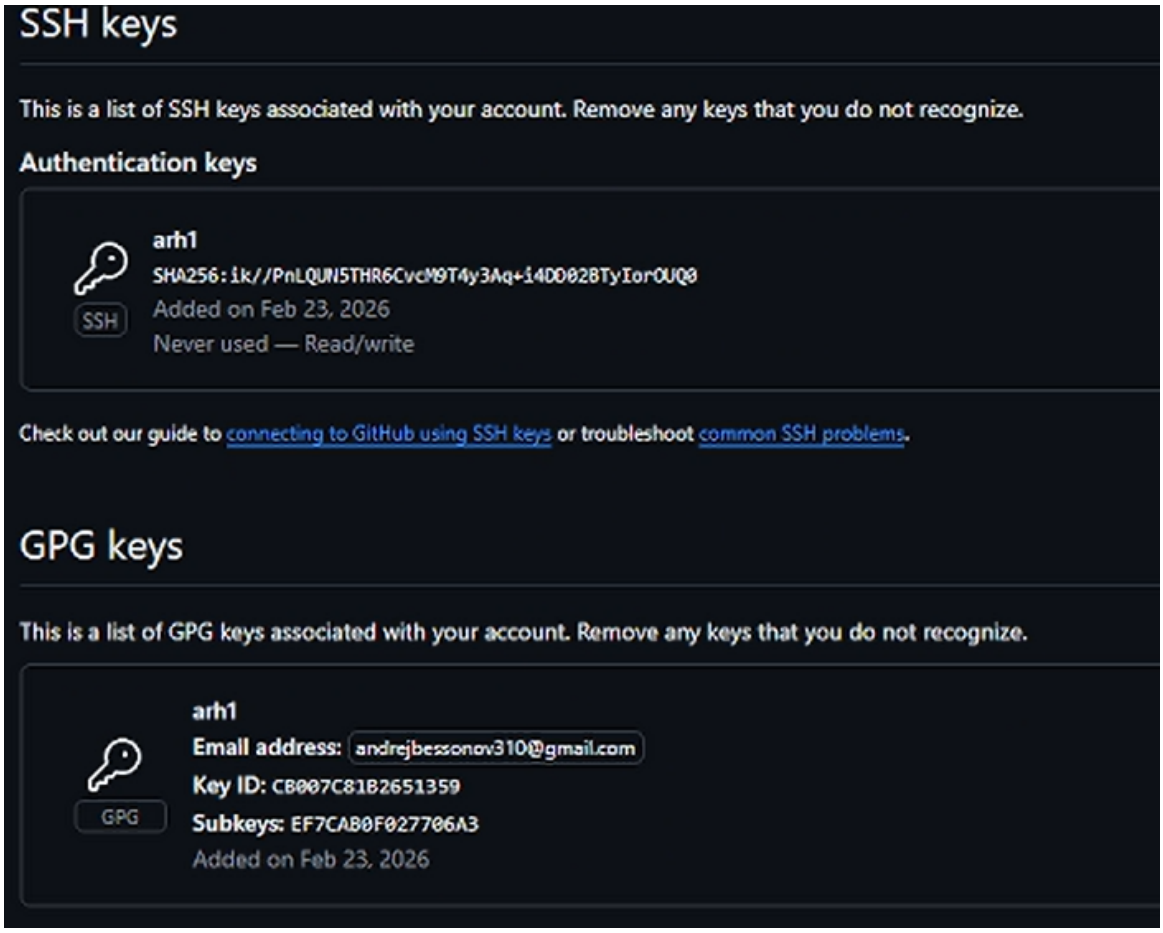

Рисунок 4.8: Рис.8

Итог: Мы полностью настроили рабочее окружение, создали ключи безопасности, связали их с GitHub и выполнили первый подписанный коммит в наш репозиторий.

5 Выводы

Изучили идеологию и применение средств контроля версий. Освоили умения по работе с git.

Список литературы